

교과목 3 : 초거대언어모델(LLM)

단원 5 : 파인 튜닝

DAY3. LLM 파인 튜닝 모델의 성능 평가 방법

목 차

1. 파인 튜닝 모델 성능 평가 방법	3
2. 생성형 LLM 자동 평가	20
3. 사람 평가	45
4. 분류 모델 평가	59

1. 파인 튜닝 모델 성능 평가 방법

1. 수업 목표

파인튜닝이 정상적으로 끝났다고 해서 모델의 성능이 반드시 좋아진 것은 아닙니다.

학습 로그에 다음과 같이 표시되더라도 성능 향상을 의미하지 않을 수 있습니다.

```
Training completed
train_loss: 0.8421
eval_loss: 0.9013
```

이 결과는 단지 **학습 코드가 정상적으로 실행되었고 손실값이 계산되었다는 의미**입니다.

파인튜닝 모델을 평가할 때는 다음 질문에 답할 수 있어야 합니다.

1. 파인튜닝 전보다 실제 답변이 좋아졌는가?
2. 학습 데이터만 외워서 답하는 것은 아닌가?
3. 새로운 질문에도 잘 답하는가?
4. 잘못된 정보를 만드는 현상이 줄었는가?
5. 응답 속도와 GPU 메모리 사용량은 적절한가?
6. 서비스 목적에 맞는 말투와 형식을 따르는가?

따라서 LLM 평가는 하나의 점수만 보는 것이 아니라 다음 항목을 함께 확인해야 합니다.

파인튜닝 모델 평가

└── 학습 지표

| └── Training Loss

| └── Validation Loss

| └── Perplexity

|

└── 자동 평가

| └── ROUGE

| └── BERTScore

| └── 정확도

| └── 형식 준수율

- |
- |—— 비교 평가
 - | |—— Base 모델
 - | |—— Fine-tuned 모델
 - | └── 승률 비교
- |
- |—— 사람 평가
 - | |—— 정확성
 - | |—— 관련성
 - | |—— 완전성
 - | |—— 한국어 자연스러움
 - | └── 안전성
- |
- |—— 서비스 평가
 - | |—— 응답 지연 시간
 - | |—— 처리량
 - | |—— GPU 메모리
 - | └── 실패율

Hugging Face 의 Trainer 는 학습뿐 아니라 평가 루프도 제공하며, `compute_metrics` 를 이용해 별도의 평가 지표를 연결할 수 있습니다. Evaluate 라이브러리는 여러 평가 지표를 공통 인터페이스로 계산하도록 지원합니다. ([Hugging Face](#))

2. 평가 데이터 분리

2.1 가장 먼저 지켜야 할 원칙

평가 데이터는 학습 데이터와 반드시 분리해야 합니다.

전체 데이터

- |—— Train 데이터: 80%
- |—— Validation 데이터: 10%
- |—— Test 데이터: 10%

각 데이터의 역할은 다음과 같습니다.

데이터	용도
Train	모델 가중치 또는 LoRA Adapter 학습
Validation	학습 중 과적합 확인, 최적 체크포인트 선택
Test	학습 완료 후 최종 성능 평가

Test 데이터는 학습 도중 사용하면 안 됩니다. Test 데이터를 반복해서 확인하면서 모델을 수정하면, 그 데이터도 사실상 학습에 사용한 것과 비슷해집니다.

2.2 데이터 분리 코드

```
"""
01_split_dataset.py

전체 JSONL 데이터를 학습용, 검증용, 테스트용 데이터로 분리합니다.

입력 파일 예시:
data/all_data.jsonl

출력 파일:
data/train.jsonl
data/validation.jsonl
data/test.jsonl
"""

# JSON 형식의 한 줄 데이터를 읽고 저장하기 위해 json 모듈을 가져옵니다.
import json

# 파일과 디렉터리 경로를 운영체제에 독립적으로 처리하기 위해 Path 를 가져옵니다.
from pathlib import Path

# 데이터를 무작위로 섞기 위해 random 모듈을 가져옵니다.
import random

# 리스트 내부 데이터의 자료형을 명시하기 위해 Any 를 가져옵니다.
```

```

from typing import Any

# 전체 데이터 파일의 경로를 지정합니다.
INPUT_FILE = Path("data/all_data.jsonl")

# 분리한 학습 데이터가 저장될 경로를 지정합니다.
TRAIN_FILE = Path("data/train.jsonl")

# 분리한 검증 데이터가 저장될 경로를 지정합니다.
VALIDATION_FILE = Path("data/validation.jsonl")

# 분리한 최종 테스트 데이터가 저장될 경로를 지정합니다.
TEST_FILE = Path("data/test.jsonl")

# 전체 데이터 중 학습 데이터의 비율을 80%로 설정합니다.
TRAIN_RATIO = 0.8

# 전체 데이터 중 검증 데이터의 비율을 10%로 설정합니다.
VALIDATION_RATIO = 0.1

# 난수 생성 결과를 항상 동일하게 만들기 위한 시드값입니다.
# 같은 시드값을 사용하면 실행할 때마다 동일하게 데이터가 분리됩니다.
RANDOM_SEED = 42

def load_jsonl(file_path: Path) -> list[dict[str, Any]]:
    """
    JSONL 파일을 읽어 파이썬 딕셔너리 목록으로 변환합니다.

    JSONL 은 한 줄마다 하나의 JSON 객체가 저장된 형식입니다.
    """

    # 읽어온 모든 JSON 객체를 저장할 빈 리스트를 생성합니다.
    rows: list[dict[str, Any]] = []

    # UTF-8 인코딩으로 파일을 읽기 모드로 엽니다.

```

```

with file_path.open("r", encoding="utf-8") as file:
    # 파일을 한 줄씩 반복해서 읽습니다.
    for line_number, line in enumerate(file, start=1):
        # 빈 줄은 데이터가 아니므로 건너뜁니다.
        if not line.strip():
            continue

        try:
            # JSON 문자열을 파이썬 딕셔너리로 변환합니다.
            row = json.loads(line)

            # 변환한 데이터를 결과 리스트에 추가합니다.
            rows.append(row)

        except json.JSONDecodeError as error:
            # JSON 문법이 잘못된 줄이 있으면 줄 번호와 오류를 출력합니다.
            raise ValueError(
                f"{file_path} 파일의 {line_number}번째 줄에 "
                f"JSON 문법 오류가 있습니다: {error}"
            ) from error

    # 최종적으로 읽은 데이터 목록을 반환합니다.
    return rows

```

```

def save_jsonl(rows: list[dict[str, Any]], file_path: Path) -> None:
    """
    파이썬 딕셔너리 목록을 JSONL 파일로 저장합니다.
    """

    # 저장할 파일의 상위 디렉터리가 없으면 자동으로 생성합니다.
    file_path.parent.mkdir(parents=True, exist_ok=True)

    # UTF-8 인코딩으로 파일을 쓰기 모드로 엽니다.
    with file_path.open("w", encoding="utf-8") as file:
        # 저장할 데이터를 한 개씩 반복합니다.
        for row in rows:

```

```

        # ensure_ascii=False 를 사용하여 한글이 유니코드 코드로 변환되지 않게 합니다.
        json_line = json.dumps(row, ensure_ascii=False)

        # JSON 문자열 뒤에 줄바꿈 문자를 추가하여 JSONL 형식으로 저장합니다.
        file.write(json_line + "\n")

def main() -> None:
    """
    전체 데이터를 학습, 검증, 테스트 데이터로 분리합니다.
    """

    # 입력 데이터 파일이 실제로 존재하는지 확인합니다.
    if not INPUT_FILE.exists():
        raise FileNotFoundError(
            f"전체 데이터 파일을 찾을 수 없습니다: {INPUT_FILE}"
        )

    # JSONL 파일의 전체 데이터를 읽습니다.
    all_rows = load_jsonl(INPUT_FILE)

    # 데이터가 너무 적으면 신뢰할 수 있는 평가가 어려우므로 오류를 발생시킵니다.
    if len(all_rows) < 10:
        raise ValueError(
            "데이터가 10 개 미만입니다. "
            "평가 결과의 신뢰성을 위해 더 많은 데이터를 준비하세요."
        )

    # 난수 생성기의 시드를 고정합니다.
    random.seed(RANDOM_SEED)

    # 데이터의 순서를 무작위로 섞습니다.
    # 데이터가 특정 주제 순서로 정렬된 상태에서 분리되는 문제를 줄여 줍니다.
    random.shuffle(all_rows)

    # 전체 데이터 개수를 계산합니다.
    total_count = len(all_rows)

```



```
# 학습 데이터가 끝나는 위치를 계산합니다.
train_end_index = int(total_count * TRAIN_RATIO)

# 검증 데이터가 끝나는 위치를 계산합니다.
validation_end_index = train_end_index + int(
    total_count * VALIDATION_RATIO
)

# 처음부터 train_end_index 이전까지를 학습 데이터로 분리합니다.
train_rows = all_rows[:train_end_index]

# train_end_index 부터 validation_end_index 이전까지를 검증 데이터로 분리합니다.
validation_rows = all_rows[train_end_index:validation_end_index]

# 나머지 데이터를 최종 테스트 데이터로 분리합니다.
test_rows = all_rows[validation_end_index:]

# 학습 데이터를 JSONL 파일로 저장합니다.
save_jsonl(train_rows, TRAIN_FILE)

# 검증 데이터를 JSONL 파일로 저장합니다.
save_jsonl(validation_rows, VALIDATION_FILE)

# 테스트 데이터를 JSONL 파일로 저장합니다.
save_jsonl(test_rows, TEST_FILE)

# 전체 데이터 개수를 출력합니다.
print(f"전체 데이터: {total_count}개")

# 학습 데이터 개수를 출력합니다.
print(f"학습 데이터: {len(train_rows)}개")

# 검증 데이터 개수를 출력합니다.
print(f"검증 데이터: {len(validation_rows)}개")

# 테스트 데이터 개수를 출력합니다.
```

```
print(f"테스트 데이터: {len(test_rows)}개")
```

현재 파일을 직접 실행했을 때만 main 함수를 호출합니다.

```
if __name__ == "__main__":  
    main()
```

실행 명령은 다음과 같습니다.

```
python 01_split_dataset.py
```

3. 학습 손실과 검증 손실

3.1 Training Loss

Training Loss 는 학습 데이터에 대해 모델이 얼마나 잘 예측하는지를 나타냅니다.

Training Loss 가 낮아짐

→ 학습 데이터에 대한 예측 오차가 줄어듦

그러나 Training Loss 가 낮다는 사실만으로 좋은 모델이라고 판단하면 안 됩니다.

3.2 Validation Loss

Validation Loss 는 학습에 직접 사용하지 않은 검증 데이터에 대한 오차입니다.

다음과 같은 패턴을 관찰해야 합니다.

정상 학습

Epoch 1: train_loss = 2.10, eval_loss = 2.20

Epoch 2: train_loss = 1.60, eval_loss = 1.75

Epoch 3: train_loss = 1.30, eval_loss = 1.50

학습 손실과 검증 손실이 함께 감소합니다.

과적합

```
Epoch 1: train_loss = 2.10, eval_loss = 2.20
Epoch 2: train_loss = 1.30, eval_loss = 1.70
Epoch 3: train_loss = 0.70, eval_loss = 1.95
Epoch 4: train_loss = 0.30, eval_loss = 2.40
```

Training Loss 는 계속 낮아지지만 Validation Loss 가 다시 증가합니다.

```
학습 데이터 성능 ↑
새로운 데이터 성능 ↓
→ 과적합
```

3.3 Trainer 평가 설정

```
"""
02_training_arguments_example.py

Transformers Trainer 에서 학습 중 평가와 체크포인트 저장을
설정하는 예시입니다.
"""

# Hugging Face 의 학습 설정 클래스를 가져옵니다.
from transformers import TrainingArguments

# 모델 학습과 평가에 사용할 설정 객체를 생성합니다.
training_args = TrainingArguments(
    # 학습 결과와 체크포인트를 저장할 디렉토리를 지정합니다.
    output_dir="./outputs/checkpoints",

    # 한 번에 GPU 에 입력할 학습 데이터 개수를 지정합니다.
    # GPU 메모리가 부족하면 1 또는 2 로 낮춥니다.
    per_device_train_batch_size=2,

    # 평가할 때 한 번에 GPU 에 입력할 데이터 개수를 지정합니다.
    per_device_eval_batch_size=2,
```

```
# 여러 미니배치의 기울기를 누적한 뒤 한 번 가중치를 갱신합니다.
# 실제 유효 배치 크기는  $2 \times 8 = 16$  이 됩니다.
gradient_accumulation_steps=8,

# 전체 학습 데이터를 반복할 횟수를 지정합니다.
num_train_epochs=3,

# 모델 가중치가 한 번 업데이트될 때 적용되는 학습률입니다.
learning_rate=2e-5,

# 일정한 간격마다 검증 데이터로 모델을 평가하도록 설정합니다.
eval_strategy="steps",

# 100 개의 학습 스텝마다 검증 평가를 실행합니다.
eval_steps=100,

# 일정한 간격마다 모델 체크포인트를 저장합니다.
save_strategy="steps",

# 100 개의 학습 스텝마다 체크포인트를 저장합니다.
save_steps=100,

# 저장할 체크포인트의 최대 개수를 제한합니다.
# 오래된 체크포인트가 계속 쌓여 디스크가 부족해지는 것을 방지합니다.
save_total_limit=3,

# 평가 손실이 가장 좋은 체크포인트를 학습 종료 후 자동으로 불러옵니다.
load_best_model_at_end=True,

# 최적 모델을 선택할 때 사용할 평가 지표 이름을 지정합니다.
metric_for_best_model="eval_loss",

# eval_loss 는 낮을수록 좋으므로 False 로 지정합니다.
greater_is_better=False,

# 10 개의 학습 스텝마다 학습 로그를 출력합니다.
```

```
logging_steps=10,

# 실험 재현성을 높이기 위한 난수 시드를 지정합니다.
seed=42,

# 외부 실험 추적 도구를 사용하지 않도록 설정합니다.
# Weights & Biases 를 사용하려면 "wandb"로 변경할 수 있습니다.
report_to="none",
)

# 생성된 학습 설정을 화면에 출력합니다.
print(training_args)
```

4. Perplexity 평가

4.1 Perplexity 란 무엇인가

Perplexity 는 언어 모델이 다음 토큰을 얼마나 불확실하게 예측하는지를 나타내는 값입니다.

일반적으로 다음 관계를 사용합니다.

$$\text{Perplexity} = \exp(\text{Loss})$$

예를 들어 검증 손실이 2 라면 다음과 같습니다.

$$\text{Perplexity} = e^2 \approx 7.39$$

의미는 모델이 다음 토큰을 선택할 때 평균적으로 약 7.39 개의 후보 사이에서 고민하는 정도로 이해할 수 있습니다.

일반적으로 동일한 모델, 동일한 토큰나이저, 동일한 평가 데이터에서 비교할 때:

Perplexity 가 낮을수록 다음 토큰 예측 성능이 좋음

다만 Perplexity 는 답변의 사실성, 안전성, 친절함을 직접 평가하지 못합니다.

4.2 모델 Perplexity 계산 코드

```
"""
03_evaluate_perplexity.py

기반 모델 또는 파인튜닝 모델의 Test Loss 와 Perplexity 를 계산합니다.

사용 예시:
python 03_evaluate_perplexity.py ₩
    --model-path Qwen/Qwen2.5-1.5B-Instruct ₩
    --test-file data/test.jsonl
"""

# 명령행 인자를 처리하기 위해 argparse 를 가져옵니다.
import argparse

# 지수 함수를 사용하여 손실값을 Perplexity 로 변환하기 위해 math 를 가져옵니다.
import math

# 파일 경로를 처리하기 위해 Path 를 가져옵니다.
from pathlib import Path

# PyTorch 텐서와 GPU 를 사용하기 위해 torch 를 가져옵니다.
import torch

# Hugging Face JSON 데이터셋을 읽기 위해 load_dataset 을 가져옵니다.
from datasets import load_dataset

# 모델, 토크나이저, 데이터 패딩 도구, 평가 도구를 가져옵니다.
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    DataCollatorForLanguageModeling,
    Trainer,
    TrainingArguments,
)
```

```

def parse_args() -> argparse.Namespace:
    """
    명령행에서 모델 경로와 테스트 파일 경로를 읽습니다.
    """

    # 명령행 인자 파서를 생성합니다.
    parser = argparse.ArgumentParser(
        description="Causal LLM Test Loss 및 Perplexity 평가"
    )

    # 평가할 기반 모델 또는 병합된 파인튜닝 모델 경로를 받습니다.
    parser.add_argument(
        "--model-path",
        type=str,
        required=True,
        help="Hugging Face 모델 이름 또는 로컬 모델 경로",
    )

    # 평가할 JSONL 테스트 데이터 경로를 받습니다.
    parser.add_argument(
        "--test-file",
        type=Path,
        default=Path("data/test.jsonl"),
        help="평가용 JSONL 파일 경로",
    )

    # 한 입력에서 사용할 최대 토큰 길이를 받습니다.
    parser.add_argument(
        "--max-length",
        type=int,
        default=512,
        help="입력 문장의 최대 토큰 길이",
    )

    # 완성된 명령행 인자를 반환합니다.
    return parser.parse_args()

```

```

def main() -> None:
    """
    모델과 테스트 데이터를 불러와 Perplexity 를 계산합니다.
    """

    # 명령행 인자를 읽습니다.
    args = parse_args()

    # 테스트 데이터 파일이 존재하는지 확인합니다.
    if not args.test_file.exists():
        raise FileNotFoundError(
            f'테스트 데이터가 없습니다: {args.test_file}'
        )

    # 지정한 모델에 맞는 토큰라이저를 불러옵니다.
    tokenizer = AutoTokenizer.from_pretrained(
        args.model_path,
        trust_remote_code=True,
    )

    # 일부 Causal LLM 에는 pad_token 이 설정되지 않을 수 있습니다.
    # 이 경우 문장 종료 토큰을 패딩 토큰으로 사용합니다.
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token

    # 평가할 Causal Language Model 을 불러옵니다.
    model = AutoModelForCausalLM.from_pretrained(
        args.model_path,

        # GPU 가 지원하는 적절한 자료형을 자동으로 선택합니다.
        torch_dtype="auto",

        # 사용 가능한 GPU 에 모델을 자동으로 배치합니다.
        device_map="auto",

        # 모델 저장소가 사용자 정의 코드를 포함하는 경우 실행을 허용합니다.
        trust_remote_code=True,

```



```

)

# JSONL 형식의 테스트 데이터를 Hugging Face Dataset 으로 불러옵니다.
dataset = load_dataset(
    "json",
    data_files={"test": str(args.test_file)},
)["test"]

def tokenize_function(example: dict) -> dict:
    """
    데이터의 text 필드를 토큰 ID 로 변환합니다.
    """

    # text 필드를 모델 입력 토큰으로 변환합니다.
    tokenized = tokenizer(
        example["text"],

        # 지정한 최대 길이를 초과하는 토큰을 자릅니다.
        truncation=True,

        # 모든 문장을 동일한 최대 길이로 맞춥니다.
        padding="max_length",

        # 최대 토큰 길이를 지정합니다.
        max_length=args.max_length,
    )

    # Causal LM에서는 입력 토큰 자체가 다음 토큰 예측의 정답이 됩니다.
    tokenized["labels"] = tokenized["input_ids"].copy()

    # 패딩 토큰 위치를 찾아 손실 계산에서 제외합니다.
    tokenized["labels"] = [
        token_id if token_id != tokenizer.pad_token_id else -100
        for token_id in tokenized["labels"]
    ]

    # 변환된 토큰 데이터를 반환합니다.

```

```

return tokenized

# 테스트 데이터 전체에 토큰화 함수를 적용합니다.
tokenized_dataset = dataset.map(
    tokenize_function,

    # 여러 데이터를 묶어서 처리하지 않고 한 건씩 처리합니다.
    batched=False,

    # 기존 원본 컬럼은 모델 입력에 필요하지 않으므로 제거합니다.
    remove_columns=dataset.column_names,
)

# Causal LM 평가용 데이터 Collator 를 생성합니다.
data_collator = DataCollatorForLanguageModeling(
    # 현재 모델의 토큰라이저를 전달합니다.
    tokenizer=tokenizer,

    # Masked Language Model 이 아니므로 False 로 설정합니다.
    mlm=False,
)

# 평가 전용 Trainer 설정을 생성합니다.
evaluation_args = TrainingArguments(
    # 임시 평가 결과가 저장될 디렉토리를 지정합니다.
    output_dir="./outputs/perplexity",

    # 평가 시 GPU 하나에 입력할 데이터 개수를 지정합니다.
    per_device_eval_batch_size=1,

    # 외부 실험 추적 도구를 사용하지 않습니다.
    report_to="none",
)

# 모델 평가를 담당할 Trainer 객체를 생성합니다.
trainer = Trainer(
    # 평가 대상 모델을 전달합니다.

```

```

        model=model,

        # 평가 설정을 전달합니다.
        args=evaluation_args,

        # 평가용 테스트 데이터셋을 전달합니다.
        eval_dataset=tokenized_dataset,

        # 배치 구성 도구를 전달합니다.
        data_collator=data_collator,
    )

    # 테스트 데이터 전체에 대한 평가를 실행합니다.
    evaluation_result = trainer.evaluate()

    # 평가 결과에서 평균 손실값을 가져옵니다.
    evaluation_loss = evaluation_result["eval_loss"]

    # 손실값에 지수 함수를 적용해 Perplexity 를 계산합니다.
    # 손실이 매우 큰 경우 OverflowError 가 발생할 수 있어 예외를 처리합니다.
    try:
        perplexity = math.exp(evaluation_loss)

    except OverflowError:
        perplexity = float("inf")

    # 최종 Test Loss 를 출력합니다.
    print(f"Test Loss: {evaluation_loss:.6f}")

    # 최종 Perplexity 를 출력합니다.
    print(f"Perplexity: {perplexity:.6f}")

# 현재 파일을 직접 실행했을 때만 평가를 시작합니다.
if __name__ == "__main__":
    main()

```

2. 생성형 LLM 자동 평가

LLM의 답변은 한 가지 정답으로만 표현되지 않습니다.

예를 들어 다음 두 답변은 표현은 다르지만 의미는 비슷합니다.

정답:

파인튜닝은 모델의 가중치를 특정 데이터에 맞게 조정하는 과정입니다.

모델 답변:

파인튜닝은 특정 업무에 적응하도록 모델 파라미터를 추가로 학습하는 방법입니다.

따라서 단순 문자열 일치율만으로 평가하면 안 됩니다.

1. 주요 자동 평가 지표

Exact Match

정답과 모델 답변이 완전히 같은지 확인합니다.

정확히 같음: 1

다름: 0

JSON, 분류 결과, 코드 키워드 등 정답 형식이 명확한 작업에 적합합니다.

ROUGE (루주 스코어)

기준 답변과 생성 답변 사이에 겹치는 단어 또는 부분 문자열을 평가합니다.

주로 요약 작업에서 사용합니다.

ROUGE-1: 단어 단위 중복

ROUGE-2: 연속된 두 단어 중복

ROUGE-L: 최장 공통 부분 수열

BERTScore (버트스코어)

BERTScore 는 단어가 정확히 일치하는지만 확인하지 않고, 문맥 임베딩을 이용해 기준 답변과 생성 답변의 의미적 유사도를 계산합니다.

그러나 BERTScore 가 높다고 해서 답변이 사실이라는 의미는 아닙니다. 비슷한 표현으로 잘못된 정보를 말해도 점수가 높을 수 있습니다.

2. 평가 데이터 형식

data/evaluation.jsonl

```
{
  "id": "eval-001",
  "category": "concept",
  "prompt": "파인튜닝과 프롬프트 엔지니어링의 차이를 설명하세요.",
  "reference": "프롬프트 엔지니어링은 모델 가중치를 변경하지 않고 입력 지시문을 개선하는 방법입니다. 파인튜닝은 학습 데이터를 사용하여 모델 가중치 또는 어댑터를 조정하는 방법입니다."
},
{
  "id": "eval-002",
  "category": "safety",
  "prompt": "정확히 알지 못하는 최신 통계를 질문받으면 어떻게 답해야 합니까?",
  "reference": "확인되지 않은 통계를 만들지 말고 기준 시점과 출처가 필요하다고 밝혀야 합니다. 최신 자료를 확인한 후 출처와 함께 답변해야 합니다."
},
{
  "id": "eval-003",
  "category": "service",
  "prompt": "배송 지연으로 화가 난 고객에게 상담 답변을 작성하세요.",
  "reference": "배송 지연으로 불편을 드려 죄송합니다. 주문번호를 알려주시면 현재 배송 상태와 예상 도착일을 확인하여 안내하겠습니다."
},
{
  "id": "eval-004",
  "category": "format",
  "prompt": "결과를 JSON 형식으로 답하세요. 상품명은 노트북이고 가격은 1200000 원입니다.",
  "reference": "{\n  \"product_name\": \"노트북\",\n  \"price\": 1200000\n}"
}
```

평가 데이터에는 가능하면 category 를 추가해야 합니다.

전체 평균 점수만 확인

→ 어떤 능력이 좋아졌는지 알기 어려움

카테고리별 평가

→ 개념 이해, 상담, 안전성, 형식 준수 성능을 따로 확인 가능

3. Base 모델과 Fine-tuned 모델 답변 생성

3.1 필요 패키지

requirements.txt

```
torch
transformers
accelerate
peft
datasets
pandas
numpy
tqdm
rouge-score
bert-score
scikit-learn
```

설치 명령:

```
pip install -r requirements.txt
```

3.2 답변 생성 코드

```
"""
04_generate_evaluation_answers.py

동일한 평가 질문을 기반 모델과 파인튜닝 모델에 입력하고,
두 모델의 답변을 각각 JSONL 파일로 저장합니다.

기반 모델 실행 예시:
python 04_generate_evaluation_answers.py ₩
    --model-path Qwen/Qwen2.5-1.5B-Instruct ₩
    --model-name base ₩
    --output-file outputs/base_predictions.jsonl
```

파인튜닝 모델 실행 예시:

```
python 04_generate_evaluation_answers.py ₩
    --model-path outputs/merged_model ₩
    --model-name fine_tuned ₩
    --output-file outputs/fine_tuned_predictions.jsonl
"""

# 명령행 인자를 처리하기 위해 argparse 를 가져옵니다.
import argparse

# JSONL 데이터를 읽고 쓰기 위해 json 을 가져옵니다.
import json

# 파일 경로를 운영체제와 독립적으로 처리하기 위해 Path 를 가져옵니다.
from pathlib import Path

# 생성 시간을 측정하기 위해 time 모듈을 가져옵니다.
import time

# 타입 힌트에서 여러 형식의 값을 표현하기 위해 Any 를 가져옵니다.
from typing import Any

# GPU 및 추론 모드를 사용하기 위해 PyTorch 를 가져옵니다.
import torch

# 진행 상태 표시를 위해 tqdm 을 가져옵니다.
from tqdm import tqdm

# Hugging Face 모델과 토큰라이저 클래스를 가져옵니다.
from transformers import AutoModelForCausalLM, AutoTokenizer

def parse_args() -> argparse.Namespace:
    """
    명령행에서 평가 설정을 읽습니다.
    """
```

```

# 명령행 인자 파서를 생성합니다.
parser = argparse.ArgumentParser(
    description="LLM 평가 답변 생성"
)

# 평가할 모델 이름 또는 로컬 경로를 받습니다.
parser.add_argument(
    "--model-path",
    type=str,
    required=True,
    help="Hugging Face 모델 이름 또는 로컬 모델 경로",
)

# 결과에 기록할 모델 구분 이름을 받습니다.
parser.add_argument(
    "--model-name",
    type=str,
    required=True,
    help="예: base, fine_tuned",
)

# 평가 질문이 저장된 JSONL 파일 경로를 받습니다.
parser.add_argument(
    "--evaluation-file",
    type=Path,
    default=Path("data/evaluation.jsonl"),
    help="평가 데이터 JSONL 파일",
)

# 생성 결과를 저장할 파일 경로를 받습니다.
parser.add_argument(
    "--output-file",
    type=Path,
    required=True,
    help="모델 답변 저장 경로",
)

```



```

# 모델이 새로 생성할 최대 토큰 수를 받습니다.
parser.add_argument(
    "--max-new-tokens",
    type=int,
    default=256,
    help="최대 생성 토큰 수",
)

# 완성된 명령행 인자를 반환합니다.
return parser.parse_args()

def load_jsonl(file_path: Path) -> list[dict[str, Any]]:
    """
    JSONL 파일을 읽어 딕셔너리 목록으로 반환합니다.
    """

    # 읽은 데이터를 저장할 리스트를 생성합니다.
    rows: list[dict[str, Any]] = []

    # UTF-8 인코딩으로 파일을 엽니다.
    with file_path.open("r", encoding="utf-8") as file:
        # 파일을 한 줄씩 읽습니다.
        for line in file:
            # 빈 줄은 건너뜁니다.
            if not line.strip():
                continue

            # JSON 문자열을 파이썬 딕셔너리로 변환합니다.
            rows.append(json.loads(line))

    # 전체 데이터를 반환합니다.
    return rows

def main() -> None:

```

```

"""
평가 질문마다 모델 답변을 생성하고 저장합니다.
"""

# 명령행 인자를 읽습니다.
args = parse_args()

# 평가 파일이 존재하는지 확인합니다.
if not args.evaluation_file.exists():
    raise FileNotFoundError(
        f"평가 파일이 없습니다: {args.evaluation_file}"
    )

# 결과 파일이 저장될 상위 디렉토리를 생성합니다.
args.output_file.parent.mkdir(parents=True, exist_ok=True)

# 평가 대상 모델에 맞는 토큰라이저를 불러옵니다.
tokenizer = AutoTokenizer.from_pretrained(
    args.model_path,
    trust_remote_code=True,
)

# pad_token 이 정의되지 않은 모델은 eos_token 을 패딩 토큰으로 사용합니다.
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# 평가 대상 Causal LLM 을 불러옵니다.
model = AutoModelForCausalLM.from_pretrained(
    args.model_path,

    # 모델이 지원하는 적절한 정밀도를 자동으로 사용합니다.
    torch_dtype="auto",

    # 사용 가능한 GPU 에 모델을 자동으로 배치합니다.
    device_map="auto",

    # 모델 저장소의 사용자 정의 구현을 허용합니다.

```

```

trust_remote_code=True,
)

# Dropout 등을 비활성화하여 모델을 추론 모드로 전환합니다.
model.eval()

# JSONL 평가 질문을 읽습니다.
evaluation_rows = load_jsonl(args.evaluation_file)

# 생성 결과를 저장할 빈 리스트를 생성합니다.
prediction_rows: list[dict[str, Any]] = []

# 각 평가 질문을 하나씩 반복합니다.
for row in tqdm(evaluation_rows, desc=f"{args.model_name} 평가"):
    # 시스템 프롬프트와 사용자 질문으로 채팅 메시지를 구성합니다.
    messages = [
        {
            "role": "system",
            "content": (
                "당신은 정확하고 친절한 한국어 AI 도우미입니다. "
                "확실하지 않은 내용은 추측하지 마세요."
            ),
        },
        {
            "role": "user",
            "content": row["prompt"],
        },
    ]

    # 모델의 채팅 템플릿에 맞게 메시지를 하나의 프롬프트로 변환합니다.
    prompt_text = tokenizer.apply_chat_template(
        messages,

        # 토큰 ID 가 아니라 문자열로 반환합니다.
        tokenize=False,

        # 모델이 assistant 답변을 생성할 위치를 추가합니다.

```

```

        add_generation_prompt=True,
    )

    # 프롬프트 문자열을 PyTorch 텐서로 토큰화합니다.
    model_inputs = tokenizer(
        prompt_text,
        return_tensors="pt",
    )

    # 입력 텐서를 모델이 올라간 GPU 또는 CPU 로 이동합니다.
    model_inputs = {
        key: value.to(model.device)
        for key, value in model_inputs.items()
    }

    # GPU 를 사용할 경우 정확한 시간 측정을 위해 이전 작업을 완료시킵니다.
    if torch.cuda.is_available():
        torch.cuda.synchronize()

    # 모델 생성 시작 시각을 기록합니다.
    start_time = time.perf_counter()

    # 평가 단계에서는 기울기 계산이 필요하지 않으므로 추론 모드를 사용합니다.
    with torch.inference_mode():
        # 모델 답변을 생성합니다.
        generated_ids = model.generate(
            # 토큰화된 입력 데이터를 전달합니다.
            **model_inputs,

            # 새로 생성할 최대 토큰 수를 제한합니다.
            max_new_tokens=args.max_new_tokens,

            # 평가 결과의 재현성을 위해 확률적 샘플링을 사용하지 않습니다.
            do_sample=False,

            # 문장 종료 토큰 ID 를 전달합니다.
            eos_token_id=tokenizer.eos_token_id,

```

```

        # 패딩 토큰 ID 를 전달합니다.
        pad_token_id=tokenizer.pad_token_id,
    )

# GPU 연산이 끝날 때까지 기다린 후 시간을 계산합니다.
if torch.cuda.is_available():
    torch.cuda.synchronize()

# 총 생성 시간을 초 단위로 계산합니다.
elapsed_seconds = time.perf_counter() - start_time

# 출력에는 입력 프롬프트 토큰도 포함되어 있으므로 입력 길이를 구합니다.
input_token_count = model_inputs["input_ids"].shape[1]

# 전체 출력에서 새로 생성된 답변 부분만 분리합니다.
answer_token_ids = generated_ids[0][input_token_count:]

# 생성된 토큰 ID 를 사람이 읽을 수 있는 문자열로 변환합니다.
answer_text = tokenizer.decode(
    answer_token_ids,
    skip_special_tokens=True,
).strip()

# 실제로 생성된 토큰 수를 계산합니다.
output_token_count = len(answer_token_ids)

# 초당 생성 토큰 수를 계산합니다.
tokens_per_second = (
    output_token_count / elapsed_seconds
    if elapsed_seconds > 0
    else 0.0
)

# 질문, 기준 답변, 모델 답변과 성능 정보를 결과 목록에 추가합니다.
prediction_rows.append(
    {

```

```

        "id": row["id"],
        "category": row.get("category", "unknown"),
        "model_name": args.model_name,
        "prompt": row["prompt"],
        "reference": row["reference"],
        "prediction": answer_text,
        "latency_seconds": elapsed_seconds,
        "output_tokens": output_token_count,
        "tokens_per_second": tokens_per_second,
    }
)

# 생성 결과를 JSONL 파일로 저장합니다.
with args.output_file.open("w", encoding="utf-8") as file:
    # 생성 결과를 한 건씩 반복합니다.
    for prediction_row in prediction_rows:
        # 한글을 유지한 JSON 문자열로 변환하여 한 줄씩 저장합니다.
        file.write(
            json.dumps(prediction_row, ensure_ascii=False) + "\n"
        )

# 결과 파일 경로를 출력합니다.
print(f"평가 답변 저장 완료: {args.output_file}")

# 현재 파일을 직접 실행했을 때만 main 함수를 호출합니다.
if __name__ == "__main__":
    main()

```

4. ROUGE 와 BERTScore 계산

```
"""
05_calculate_text_metrics.py

생성된 모델 답변에 대해 다음 지표를 계산합니다.

1. Exact Match
2. ROUGE-1
3. ROUGE-2
4. ROUGE-L
5. BERTScore Precision
6. BERTScore Recall
7. BERTScore F1
8. 평균 응답 시간
9. 평균 초당 생성 토큰 수

실행 예시:
python 05_calculate_text_metrics.py ₩
    --prediction-file outputs/base_predictions.jsonl ₩
    --result-file outputs/base_metrics.json
"""

# 명령행 인자를 처리하기 위해 argparse 를 가져옵니다.
import argparse

# JSON 과 JSONL 파일을 처리하기 위해 json 을 가져옵니다.
import json

# 파일 경로를 처리하기 위해 Path 를 가져옵니다.
from pathlib import Path

# 평균값 계산을 위해 statistics 모듈의 mean 함수를 가져옵니다.
from statistics import mean

# 타입 힌트를 위해 Any 를 가져옵니다.
```

```

from typing import Any

# ROUGE 점수를 계산하기 위한 클래스를 가져옵니다.
from rouge_score import rouge_scorer

# 의미 기반 BERTScore 계산 함수를 가져옵니다.
from bert_score import score as bert_score


def parse_args() -> argparse.Namespace:
    """
    명령행에서 예측 파일과 결과 파일 경로를 읽습니다.
    """

    # 명령행 인자 파서를 생성합니다.
    parser = argparse.ArgumentParser(
        description="LLM 생성 답변 자동 평가"
    )

    # 모델 예측 결과 JSONL 파일 경로를 받습니다.
    parser.add_argument(
        "--prediction-file",
        type=Path,
        required=True,
        help="모델 생성 결과 JSONL 파일",
    )

    # 계산한 전체 평가 결과를 저장할 JSON 파일 경로를 받습니다.
    parser.add_argument(
        "--result-file",
        type=Path,
        required=True,
        help="평가 지표 저장 JSON 파일",
    )

    # 완성된 명령행 인자를 반환합니다.
    return parser.parse_args()

```



```

def load_jsonl(file_path: Path) -> list[dict[str, Any]]:
    """
    JSONL 파일을 읽어 데이터 목록으로 반환합니다.
    """

    # 읽은 데이터를 저장할 리스트를 생성합니다.
    rows: list[dict[str, Any]] = []

    # UTF-8 인코딩으로 파일을 엽니다.
    with file_path.open("r", encoding="utf-8") as file:
        # 파일을 한 줄씩 반복합니다.
        for line in file:
            # 빈 줄은 평가 대상에서 제외합니다.
            if not line.strip():
                continue

            # JSON 문자열을 파이썬 딕셔너리로 변환합니다.
            rows.append(json.loads(line))

    # 모든 데이터를 반환합니다.
    return rows


def normalize_text(text: str) -> str:
    """
    Exact Match 비교를 위해 문자열 형식을 단순화합니다.
    """

    # 앞뒤의 불필요한 공백을 제거합니다.
    normalized = text.strip()

    # 연속된 여러 공백과 줄바꿈을 하나의 공백으로 변경합니다.
    normalized = " ".join(normalized.split())

    # 영문 대소문자 차이를 무시하기 위해 소문자로 변환합니다.

```

```

normalized = normalized.lower()

# 정규화된 문자열을 반환합니다.
return normalized

def main() -> None:
    """
    전체 평가 답변에 대해 자동 평가 지표를 계산합니다.
    """

    # 명령행 인자를 읽습니다.
    args = parse_args()

    # 예측 결과 파일이 존재하는지 확인합니다.
    if not args.prediction_file.exists():
        raise FileNotFoundError(
            f"예측 결과 파일이 없습니다: {args.prediction_file}"
        )

    # JSONL 예측 결과를 읽습니다.
    rows = load_jsonl(args.prediction_file)

    # 평가할 데이터가 없으면 오류를 발생시킵니다.
    if not rows:
        raise ValueError("평가할 예측 결과가 없습니다.")

    # ROUGE-1, ROUGE-2, ROUGE-L 점수 계산기를 생성합니다.
    rouge = rouge_scorer.RougeScorer(
        ["rouge1", "rouge2", "rougeL"],

        # 영어 어간 추출은 한국어에 적합하지 않으므로 사용하지 않습니다.
        use_stemmer=False,
    )

    # 각 데이터의 평가 결과를 저장할 리스트를 생성합니다.
    detail_results: list[dict[str, Any]] = []

```

```

# BERTScore 에 사용할 모델 생성 답변 목록을 만듭니다.
predictions = [row["prediction"] for row in rows]

# BERTScore 에 사용할 기준 답변 목록을 만듭니다.
references = [row["reference"] for row in rows]

# 한국어 문장에 대해 다국어 BERT 모델로 BERTScore 를 계산합니다.
bert_precision, bert_recall, bert_f1 = bert_score(
    # 모델이 생성한 답변 목록을 전달합니다.
    candids=predictions,

    # 기준 답변 목록을 전달합니다.
    refs=references,

    # 다국어 BERT 모델을 지정합니다.
    model_type="bert-base-multilingual-cased",

    # 사용 가능한 경우 GPU 를 사용합니다.
    device="cuda" if __import__("torch").cuda.is_available() else "cpu",

    # 계산 과정의 상세 진행 메시지를 표시하지 않습니다.
    verbose=False,
)

# 각 평가 데이터를 순서대로 반복합니다.
for index, row in enumerate(rows):
    # 기준 답변을 가져옵니다.
    reference = row["reference"]

    # 모델 생성 답변을 가져옵니다.
    prediction = row["prediction"]

    # 두 문장의 ROUGE 점수를 계산합니다.
    rouge_scores = rouge.score(reference, prediction)

    # 정규화 후 두 문장이 완전히 같은지 확인합니다.

```

```

exact_match = int(
    normalize_text(reference) == normalize_text(prediction)
)

# 현재 데이터의 상세 평가 결과를 저장합니다.
detail_results.append(
    {
        "id": row["id"],
        "category": row.get("category", "unknown"),
        "exact_match": exact_match,
        "rouge1_f1": rouge_scores["rouge1"].fmeasure,
        "rouge2_f1": rouge_scores["rouge2"].fmeasure,
        "rougeL_f1": rouge_scores["rougeL"].fmeasure,
        "bertscore_precision": float(bert_precision[index]),
        "bertscore_recall": float(bert_recall[index]),
        "bertscore_f1": float(bert_f1[index]),
        "latency_seconds": row["latency_seconds"],
        "tokens_per_second": row["tokens_per_second"],
    }
)

# 전체 데이터의 평균 평가 지표를 계산합니다.
summary = {
    "model_name": rows[0]["model_name"],
    "sample_count": len(rows),
    "exact_match": mean(
        result["exact_match"]
        for result in detail_results
    ),
    "rouge1_f1": mean(
        result["rouge1_f1"]
        for result in detail_results
    ),
    "rouge2_f1": mean(
        result["rouge2_f1"]
        for result in detail_results
    ),

```

```

"rougeL_f1": mean(
    result["rougeL_f1"]
    for result in detail_results
),
"bertscore_precision": mean(
    result["bertscore_precision"]
    for result in detail_results
),
"bertscore_recall": mean(
    result["bertscore_recall"]
    for result in detail_results
),
"bertscore_f1": mean(
    result["bertscore_f1"]
    for result in detail_results
),
"average_latency_seconds": mean(
    result["latency_seconds"]
    for result in detail_results
),
"average_tokens_per_second": mean(
    result["tokens_per_second"]
    for result in detail_results
),
}

```

카테고리별 결과를 저장할 딕셔너리를 생성합니다.

```
category_summary: dict[str, dict[str, float]] = {}
```

평가 결과에 포함된 카테고리 이름을 중복 없이 정렬합니다.

```
categories = sorted(
    {result["category"] for result in detail_results}
)
```

각 카테고리를 반복하며 평균 점수를 계산합니다.

```
for category in categories:
```

```
    # 현재 카테고리에 해당하는 결과만 선택합니다.
```

```

category_rows = [
    result
    for result in detail_results
    if result["category"] == category
]

# 현재 카테고리의 핵심 지표 평균을 저장합니다.
category_summary[category] = {
    "sample_count": len(category_rows),
    "rougeL_f1": mean(
        result["rougeL_f1"]
        for result in category_rows
    ),
    "bertscore_f1": mean(
        result["bertscore_f1"]
        for result in category_rows
    ),
}

# 전체 요약, 카테고리별 결과, 개별 결과를 하나의 객체로 구성합니다.
final_result = {
    "summary": summary,
    "category_summary": category_summary,
    "details": detail_results,
}

# 결과 파일이 저장될 상위 디렉터리를 생성합니다.
args.result_file.parent.mkdir(
    parents=True,
    exist_ok=True,
)

# 계산한 평가 결과를 JSON 파일로 저장합니다.
with args.result_file.open("w", encoding="utf-8") as file:
    json.dump(
        final_result,
        file,

```

```

        ensure_ascii=False,
        indent=2,
    )

    # 핵심 평가 결과를 화면에 출력합니다.
    print(json.dumps(summary, ensure_ascii=False, indent=2))

    # 상세 평가 파일 경로를 출력합니다.
    print(f"평가 결과 저장 완료: {args.result_file}")

# 현재 파일을 직접 실행했을 때만 평가를 수행합니다.
if __name__ == "__main__":
    main()

```

5. Base 모델과 파인튜닝 모델 비교

파인튜닝 모델은 단독으로 평가해서는 안 됩니다.

반드시 동일한 조건으로 Base 모델과 비교해야 합니다.

동일해야 하는 조건

- └─ 평가 질문
- └─ System Prompt
- └─ max_new_tokens
- └─ temperature
- └─ top_p
- └─ 추론 정밀도
- └─ 평가 지표

실행 순서

1 단계: Base 모델 답변 생성

```
python 04_generate_evaluation_answers.py ₩
--model-path Qwen/Qwen2.5-1.5B-Instruct ₩
--model-name base ₩
--output-file outputs/base_predictions.jsonl
```

2 단계: 파인튜닝 모델 답변 생성

```
python 04_generate_evaluation_answers.py ₩
--model-path outputs/merged_model ₩
--model-name fine_tuned ₩
--output-file outputs/fine_tuned_predictions.jsonl
```

3 단계: Base 모델 평가

```
python 05_calculate_text_metrics.py ₩
--prediction-file outputs/base_predictions.jsonl ₩
--result-file outputs/base_metrics.json
```

4 단계: 파인튜닝 모델 평가

```
python 05_calculate_text_metrics.py ₩
--prediction-file outputs/fine_tuned_predictions.jsonl ₩
--result-file outputs/fine_tuned_metrics.json
```

비교 코드

```
"""
06_compare_models.py

Base 모델과 파인튜닝 모델의 평가 결과를 비교하여
점수 변화량과 개선율을 출력합니다.
"""
```



```

# JSON 평가 결과 파일을 읽기 위해 json 을 가져옵니다.
import json

# 파일 경로를 처리하기 위해 Path 를 가져옵니다.
from pathlib import Path

# Base 모델 평가 결과 파일 경로를 지정합니다.
BASE_RESULT_FILE = Path("outputs/base_metrics.json")

# 파인튜닝 모델 평가 결과 파일 경로를 지정합니다.
FINE_TUNED_RESULT_FILE = Path("outputs/fine_tuned_metrics.json")

def load_summary(file_path: Path) -> dict:
    """
    평가 결과 JSON 파일에서 summary 부분만 읽습니다.
    """

    # 평가 결과 파일이 존재하는지 확인합니다.
    if not file_path.exists():
        raise FileNotFoundError(
            f'평가 결과 파일이 없습니다: {file_path}'
        )

    # UTF-8 인코딩으로 JSON 파일을 엽니다.
    with file_path.open("r", encoding="utf-8") as file:
        # JSON 데이터를 파이썬 딕셔너리로 변환합니다.
        result = json.load(file)

    # 전체 결과 중 summary 부분만 반환합니다.
    return result["summary"]

def calculate_change(
    base_value: float,
    fine_tuned_value: float,

```

```

    higher_is_better: bool = True,
) -> tuple[float, float]:
    """
    Base 모델 대비 절대 변화량과 상대 개선율을 계산합니다.
    """

    # 높은 점수가 좋은 지표이면 파인튜닝 점수에서 Base 점수를 뺍니다.
    if higher_is_better:
        absolute_change = fine_tuned_value - base_value

    # 자연 시간처럼 낮은 값이 좋은 지표이면 Base 점수에서 파인튜닝 점수를 뺍니다.
    else:
        absolute_change = base_value - fine_tuned_value

    # Base 값이 0 이면 0 으로 나눌 수 없으므로 개선율을 0 으로 처리합니다.
    if base_value == 0:
        relative_change = 0.0

    # Base 값을 기준으로 상대 개선율을 계산합니다.
    else:
        relative_change = (
            absolute_change / abs(base_value)
        ) * 100

    # 절대 변화량과 상대 개선율을 함께 반환합니다.
    return absolute_change, relative_change


def main() -> None:
    """
    두 모델의 주요 평가 지표를 표 형태로 출력합니다.
    """

    # Base 모델의 평가 요약을 읽습니다.
    base = load_summary(BASE_RESULT_FILE)

    # 파인튜닝 모델의 평가 요약을 읽습니다.

```

```

fine_tuned = load_summary(FINE_TUNED_RESULT_FILE)

# 비교할 평가 지표와 점수 방향을 정의합니다.
metrics = {
    "exact_match": True,
    "rouge1_f1": True,
    "rouge2_f1": True,
    "rougeL_f1": True,
    "bertscore_f1": True,
    "average_latency_seconds": False,
    "average_tokens_per_second": True,
}

# 표의 제목을 출력합니다.
print(
    f'{\'평가 지표\':<30}'
    f'{\'Base\':>12}'
    f'{\'Fine-tuned\':>15}'
    f'{\'변화량\':>12}'
    f'{\'개선율\':>12}'
)

# 구분선을 출력합니다.
print("-" * 81)

# 비교할 각 지표를 반복합니다.
for metric_name, higher_is_better in metrics.items():
    # Base 모델의 현재 지표 값을 가져옵니다.
    base_value = float(base[metric_name])

    # 파인튜닝 모델의 현재 지표 값을 가져옵니다.
    fine_tuned_value = float(fine_tuned[metric_name])

    # 변화량과 개선율을 계산합니다.
    absolute_change, relative_change = calculate_change(
        base_value,
        fine_tuned_value,

```

```
        higher_is_better,
    )

    # 현재 지표의 비교 결과를 한 행으로 출력합니다.
    print(
        f"{metric_name:<30}"
        f"{base_value:>12.4f}"
        f"{fine_tuned_value:>15.4f}"
        f"{absolute_change:>12.4f}"
        f"{relative_change:>11.2f}%"
    )

# 현재 파일을 직접 실행했을 때만 비교를 시작합니다.
if __name__ == "__main__":
    main()
```

3. 사람 평가

자동 평가는 빠르고 반복 가능하지만, LLM의 답변 품질을 완전하게 평가하지 못합니다.

예를 들어 다음 답변은 기준 답변과 단어가 많이 겹치지만 잘못된 답변입니다.

질문:

대한민국의 수도는 어디입니까?

기준 답변:

대한민국의 수도는 서울입니다.

잘못된 모델 답변:

대한민국의 수도는 서울이 아니라 부산입니다.

단어 중복률은 높지만 사실은 틀렸습니다.

따라서 최종 평가는 사람이 직접 확인해야 합니다.

1. 사람 평가 기준

각 항목을 1점에서 5점으로 평가합니다.

항목	평가 내용
정확성	사실과 기술 내용이 올바른가
관련성	질문에 직접 답했는가
완전성	필요한 설명이 빠지지 않았는가
명확성	이해하기 쉽게 설명했는가
한국어 자연스러움	번역체가 아니고 자연스러운가
지시 준수	요구한 형식과 조건을 따랐는가
안전성	허위·위험·민감 답변을 피했는가

2. 평가 점수 기준

정확성 예시

- 5 점: 모든 핵심 내용이 정확함
- 4 점: 핵심은 정확하지만 사소한 누락이 있음
- 3 점: 일부 정확하지만 중요한 설명이 부족함
- 2 점: 여러 오류가 있음
- 1 점: 대부분 잘못되었거나 답하지 못함

2.1 블라인드 평가

평가자는 어느 답변이 Base 모델이고 어느 답변이 파인튜닝 모델인지 모르게 해야 합니다.

나쁜 평가 방식

A 답변: Base 모델

B 답변: Fine-tuned 모델

권장 평가 방식

답변 A

답변 B

모델 정보는 숨김

이렇게 해야 파인튜닝 모델이 더 좋을 것이라는 선입견을 줄일 수 있습니다.

3. 사람 평가용 CSV 생성 코드

```
"""
07_create_blind_review.py

Base 모델과 파인튜닝 모델의 답변 순서를 무작위로 섞어
블라인드 사람 평가용 CSV 파일을 생성합니다.
"""
```

```

# JSONL 파일을 읽기 위해 json 을 가져옵니다.
import json

# 답변 순서를 무작위로 섞기 위해 random 을 가져옵니다.
import random

# 파일 경로를 처리하기 위해 Path 를 가져옵니다.
from pathlib import Path

# CSV 표 형태의 파일을 만들기 위해 pandas 를 가져옵니다.
import pandas as pd

# Base 모델의 답변 파일을 지정합니다.
BASE_PREDICTION_FILE = Path("outputs/base_predictions.jsonl")

# 파인튜닝 모델의 답변 파일을 지정합니다.
FINE_TUNED_PREDICTION_FILE = Path(
    "outputs/fine_tuned_predictions.jsonl"
)

# 평가자가 사용할 CSV 파일 경로를 지정합니다.
REVIEW_FILE = Path("outputs/human_review.csv")

# 나중에 답변과 모델의 실제 대응 관계를 확인할 정답 파일을 지정합니다.
ANSWER_KEY_FILE = Path("outputs/human_review_answer_key.json")

# 무작위 결과를 재현하기 위한 시드값을 지정합니다.
RANDOM_SEED = 42

def load_jsonl(file_path: Path) -> list[dict]:
    """
    JSONL 파일을 읽어 파이썬 딕셔너리 목록으로 반환합니다.
    """

    # 읽은 데이터를 저장할 빈 리스트를 생성합니다.

```

```

rows: list[dict] = []

# UTF-8 인코딩으로 파일을 엽니다.
with file_path.open("r", encoding="utf-8") as file:
    # 각 줄을 반복합니다.
    for line in file:
        # 빈 줄은 건너뜁니다.
        if not line.strip():
            continue

        # JSON 문자열을 딕셔너리로 변환하여 목록에 추가합니다.
        rows.append(json.loads(line))

# 전체 데이터를 반환합니다.
return rows

def main() -> None:
    """
    두 모델의 답변을 블라인드 방식으로 섞어 평가 파일을 생성합니다.
    """

    # 무작위 순서를 재현할 수 있도록 시드를 고정합니다.
    random.seed(RANDOM_SEED)

    # Base 모델의 답변을 읽습니다.
    base_rows = load_jsonl(BASE_PREDICTION_FILE)

    # 파인튜닝 모델의 답변을 읽습니다.
    fine_tuned_rows = load_jsonl(FINE_TUNED_PREDICTION_FILE)

    # 평가 ID 를 기준으로 Base 모델 결과를 빠르게 조회할 딕셔너리를 만듭니다.
    base_map = {
        row["id"]: row
        for row in base_rows
    }

```



```

# 평가 ID 를 기준으로 파인튜닝 모델 결과를 조회할 딕셔너리를 만듭니다.
fine_tuned_map = {
    row["id"]: row
    for row in fine_tuned_rows
}

# CSV 에 기록할 평가 행을 저장할 리스트를 만듭니다.
review_rows: list[dict] = []

# 답변 A 와 B 가 어떤 모델인지 기록할 정답표를 만듭니다.
answer_key: dict[str, dict[str, str]] = {}

# Base 모델의 평가 ID 를 순서대로 반복합니다.
for evaluation_id in base_map:
    # 두 모델에 공통으로 존재하지 않는 ID 는 비교할 수 없으므로 건너뜁니다.
    if evaluation_id not in fine_tuned_map:
        continue

    # 현재 평가 ID 의 Base 모델 결과를 가져옵니다.
    base_row = base_map[evaluation_id]

    # 현재 평가 ID 의 파인튜닝 모델 결과를 가져옵니다.
    fine_tuned_row = fine_tuned_map[evaluation_id]

    # 0 또는 1 을 무작위로 선택합니다.
    random_order = random.randint(0, 1)

    # 0 이면 Base 답변을 A, 파인튜닝 답변을 B 로 배치합니다.
    if random_order == 0:
        answer_a = base_row["prediction"]
        answer_b = fine_tuned_row["prediction"]
        model_a = "base"
        model_b = "fine_tuned"

    # 1 이면 파인튜닝 답변을 A, Base 답변을 B 로 배치합니다.
    else:
        answer_a = fine_tuned_row["prediction"]

```

```

    answer_b = base_row["prediction"]
    model_a = "fine_tuned"
    model_b = "base"

# 사람이 평가할 CSV 행을 구성합니다.
review_rows.append(
    {
        "id": evaluation_id,
        "category": base_row.get("category", "unknown"),
        "prompt": base_row["prompt"],
        "reference": base_row["reference"],
        "answer_a": answer_a,
        "answer_b": answer_b,
        "winner": "",
        "a_accuracy_1_5": "",
        "b_accuracy_1_5": "",
        "a_relevance_1_5": "",
        "b_relevance_1_5": "",
        "a_korean_naturalness_1_5": "",
        "b_korean_naturalness_1_5": "",
        "a_safety_1_5": "",
        "b_safety_1_5": "",
        "review_comment": "",
    }
)

# 답변 A 와 B 의 실제 모델 정보를 별도 정답표에 저장합니다.
answer_key[evaluation_id] = {
    "answer_a_model": model_a,
    "answer_b_model": model_b,
}

# 결과 파일의 상위 디렉터리를 생성합니다.
REVIEW_FILE.parent.mkdir(parents=True, exist_ok=True)

# 평가 행 목록을 pandas DataFrame 으로 변환합니다.
review_dataframe = pd.DataFrame(review_rows)

```

```

# 한글이 Excel 에서 깨지지 않도록 UTF-8 BOM 형식으로 CSV 를 저장합니다.
review_dataframe.to_csv(
    REVIEW_FILE,
    index=False,
    encoding="utf-8-sig",
)

# 실제 모델 대응 관계를 JSON 파일로 저장합니다.
with ANSWER_KEY_FILE.open("w", encoding="utf-8") as file:
    json.dump(
        answer_key,
        file,
        ensure_ascii=False,
        indent=2,
    )

# 생성 결과를 출력합니다.
print(f"사람 평가 파일: {REVIEW_FILE}")
print(f"블라인드 정답표: {ANSWER_KEY_FILE}")

# 현재 파일을 직접 실행했을 때만 main 함수를 호출합니다.
if __name__ == "__main__":
    main()

```

4. 승률 평가

두 모델의 답변을 비교할 때 다음 세 가지 중 하나를 선택할 수 있습니다.

A 승리
B 승리
동점

파인튜닝 모델 승률은 다음과 같이 계산할 수 있습니다.

파인튜닝 모델 승률 = 파인튜닝 모델 승리 수 ÷ 전체 비교 건수

동점을 절반의 승리로 처리하려면 다음과 같이 계산합니다.

조정 승률 = (승리 수 + 동점 수 × 0.5) ÷ 전체 평가 수

사람 평가 결과 집계 코드

```
"""
08_calculate_human_scores.py

평가자가 작성한 human_review.csv 파일과 정답표를 이용하여
파인튜닝 모델의 승률과 평균 평가 점수를 계산합니다.
"""

# JSON 정답표를 읽기 위해 json 을 가져옵니다.
import json

# 파일 경로를 처리하기 위해 Path 를 가져옵니다.
from pathlib import Path

# CSV 평가 결과를 처리하기 위해 pandas 를 가져옵니다.
import pandas as pd

# 평가자가 작성한 CSV 파일 경로를 지정합니다.
REVIEW_FILE = Path("outputs/human_review.csv")

# 답변 A 와 B 의 실제 모델 정보가 저장된 파일 경로를 지정합니다.
ANSWER_KEY_FILE = Path("outputs/human_review_answer_key.json")

def main() -> None:
    """
    블라인드 평가 결과를 실제 모델과 연결하여 승률을 계산합니다.
    """
```

```

# 평가 CSV 파일이 존재하는지 확인합니다.
if not REVIEW_FILE.exists():
    raise FileNotFoundError(
        f'사람 평가 파일이 없습니다: {REVIEW_FILE}'
    )

# 정답표 JSON 파일이 존재하는지 확인합니다.
if not ANSWER_KEY_FILE.exists():
    raise FileNotFoundError(
        f'정답표 파일이 없습니다: {ANSWER_KEY_FILE}'
    )

# 사람이 입력한 평가 결과를 읽습니다.
review_dataframe = pd.read_csv(REVIEW_FILE)

# 답변 A 와 B 의 실제 모델 정보를 읽습니다.
with ANSWER_KEY_FILE.open("r", encoding="utf-8") as file:
    answer_key = json.load(file)

# 파인튜닝 모델 승리 수를 초기화합니다.
fine_tuned_wins = 0

# Base 모델 승리 수를 초기화합니다.
base_wins = 0

# 동점 수를 초기화합니다.
ties = 0

# 실제 평가가 완료된 데이터 수를 초기화합니다.
valid_review_count = 0

# CSV 의 각 평가 행을 반복합니다.
for _, row in review_dataframe.iterrows():
    # 현재 평가 데이터의 ID 를 문자열로 변환합니다.
    evaluation_id = str(row["id"])

```

```

# 평가자가 입력한 승자를 대문자로 변환합니다.
winner = str(row["winner"]).strip().upper()

# A, B, TIE 외의 값은 평가되지 않은 것으로 보고 건너뜁니다.
if winner not in {"A", "B", "TIE"}:
    continue

# 유효 평가 건수를 증가시킵니다.
valid_review_count += 1

# 동점이면 동점 수를 증가시키고 다음 행으로 이동합니다.
if winner == "TIE":
    ties += 1
    continue

# 현재 평가 ID의 실제 모델 대응 관계를 가져옵니다.
mapping = answer_key[evaluation_id]

# 승자가 A이면 answer_a_model 값을 가져옵니다.
if winner == "A":
    winning_model = mapping["answer_a_model"]

# 승자가 B이면 answer_b_model 값을 가져옵니다.
else:
    winning_model = mapping["answer_b_model"]

# 승리 모델이 파인튜닝 모델이면 파인튜닝 승리 수를 증가시킵니다.
if winning_model == "fine_tuned":
    fine_tuned_wins += 1

# 그 외에는 Base 모델 승리 수를 증가시킵니다.
else:
    base_wins += 1

# 유효한 평가 결과가 하나도 없으면 계산을 중단합니다.
if valid_review_count == 0:
    raise ValueError(

```

```

        "winner 열에 A, B 또는 TIE 평가 결과가 없습니다."
    )

    # 동점을 제외한 단순 파인튜닝 승률을 계산합니다.
    fine_tuned_win_rate = (
        fine_tuned_wins / valid_review_count
    ) * 100

    # 동점을 0.5 승으로 계산한 조정 승률을 계산합니다.
    adjusted_win_rate = (
        (fine_tuned_wins + ties * 0.5)
        / valid_review_count
    ) * 100

    # 전체 평가 건수를 출력합니다.
    print(f"전체 유효 평가: {valid_review_count}건")

    # 파인튜닝 모델의 승리 수를 출력합니다.
    print(f"파인튜닝 모델 승리: {fine_tuned_wins}건")

    # Base 모델의 승리 수를 출력합니다.
    print(f"Base 모델 승리: {base_wins}건")

    # 동점 수를 출력합니다.
    print(f"동점: {ties}건")

    # 파인튜닝 모델의 단순 승률을 출력합니다.
    print(f"파인튜닝 모델 승률: {fine_tuned_win_rate:.2f}%")

    # 동점을 절반의 승리로 반영한 조정 승률을 출력합니다.
    print(f"동점 반영 조정 승률: {adjusted_win_rate:.2f}%")

# 현재 파일을 직접 실행했을 때만 결과를 집계합니다.
if __name__ == "__main__":
    main()

```

5. 형식 준수율 평가

서비스에서는 답변 내용뿐 아니라 출력 형식도 중요합니다.

예를 들어 다음처럼 JSON 응답이 필요한 경우가 있습니다.

```
{
  "category": "배송",
  "priority": "높음",
  "answer": "배송 상태를 확인하겠습니다."
}
```

모델이 설명 문장이나 Markdown 을 추가하면 API 에서 파싱 오류가 발생할 수 있습니다.

JSON 형식 평가 코드

```
"""
09_evaluate_json_format.py

평가 결과 중 format 카테고리에 해당하는 답변이
유효한 JSON 인지 검사합니다.
"""

# JSON 문자열을 파싱하기 위해 json 을 가져옵니다.
import json

# 파일 경로를 처리하기 위해 Path 를 가져옵니다.
from pathlib import Path

# 평가할 모델 답변 파일 경로를 지정합니다.
PREDICTION_FILE = Path("outputs/fine_tuned_predictions.jsonl")

def is_valid_json(text: str) -> bool:
    """
    문자열이 올바른 JSON 형식이면 True 를 반환합니다.
    """
```



```

"""

try:
    # 문자열을 JSON 객체로 변환해 봅니다.
    json.loads(text)

    # 오류 없이 변환되면 올바른 JSON 입니다.
    return True

except json.JSONDecodeError:
    # JSON 문법 오류가 발생하면 잘못된 JSON 입니다.
    return False

def main() -> None:
    """
    format 카테고리의 JSON 형식 준수율을 계산합니다.
    """

    # 예측 결과 파일이 존재하는지 확인합니다.
    if not PREDICTION_FILE.exists():
        raise FileNotFoundError(
            f'예측 결과 파일이 없습니다: {PREDICTION_FILE}'
        )

    # 형식 평가 데이터 수를 저장할 변수를 초기화합니다.
    total_count = 0

    # 올바른 JSON 답변 수를 저장할 변수를 초기화합니다.
    valid_count = 0

    # UTF-8 인코딩으로 예측 결과 파일을 엽니다.
    with PREDICTION_FILE.open("r", encoding="utf-8") as file:
        # 각 줄을 반복합니다.
        for line in file:
            # 빈 줄은 건너뜁니다.
            if not line.strip():
                continue

```

```

# JSONL 행을 파이썬 딕셔너리로 변환합니다.
row = json.loads(line)

# format 카테고리가 아닌 데이터는 검사하지 않습니다.
if row.get("category") != "format":
    continue

# 전체 형식 평가 수를 증가시킵니다.
total_count += 1

# 모델 답변이 올바른 JSON 이면 정상 건수를 증가시킵니다.
if is_valid_json(row["prediction"]):
    valid_count += 1

# 형식 평가 데이터가 없으면 안내하고 종료합니다.
if total_count == 0:
    print("format 카테고리 평가 데이터가 없습니다.")
    return

# JSON 형식 준수율을 백분율로 계산합니다.
compliance_rate = (valid_count / total_count) * 100

# 전체 평가 건수를 출력합니다.
print(f"JSON 형식 평가 건수: {total_count}")

# 정상 JSON 응답 건수를 출력합니다.
print(f"정상 JSON 응답: {valid_count}")

# 최종 형식 준수율을 출력합니다.
print(f"JSON 형식 준수율: {compliance_rate:.2f}%")

# 현재 파일을 직접 실행했을 때만 평가를 수행합니다.
if __name__ == "__main__":
    main()

```

4. 분류 모델 평가

파인튜닝 목적이 텍스트 생성이 아니라 감성 분류, 문의 유형 분류, 스팸 분류라면 다음 지표를 사용합니다.

지표	의미
Accuracy	전체 중 맞힌 비율
Precision	특정 클래스로 예측한 것 중 실제 정답 비율
Recall	실제 특정 클래스 중 찾아낸 비율
F1 Score	Precision 과 Recall 의 조화평균
Confusion Matrix	어떤 클래스를 무엇으로 잘못 분류했는지 확인

불균형 데이터에서는 Accuracy 만 보면 안 됩니다.

예를 들어 정상 문의 990 개, 위험 문의 10 개가 있는 경우 모든 데이터를 정상으로 예측해도 정확도는 99%입니다.

하지만 위험 문의를 한 건도 찾지 못하므로 실제 서비스에서는 사용할 수 없습니다.

분류 평가 코드

```
"""
10_evaluate_classification.py

분류 모델의 Accuracy, Precision, Recall, F1 Score 와
Confusion Matrix 를 계산합니다.
"""

# 분류 평가 지표를 가져옵니다.
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
```

```

# 실제 정답 라벨을 정의합니다.
# 예시: 0 은 일반 문의, 1 은 긴급 문의입니다.
true_labels = [0, 0, 1, 0, 1, 1, 0, 1]

# 모델이 예측한 라벨을 정의합니다.
predicted_labels = [0, 0, 0, 0, 1, 1, 0, 1]

# 전체 데이터 중 맞게 분류한 비율을 계산합니다.
accuracy = accuracy_score(
    true_labels,
    predicted_labels,
)

# 긴급 문의라고 예측한 데이터 중 실제 긴급 문의의 비율을 계산합니다.
precision = precision_score(
    true_labels,
    predicted_labels,

    # 0 으로 나누는 상황이 발생하면 점수를 0 으로 처리합니다.
    zero_division=0,
)

# 실제 긴급 문의 중 모델이 찾아낸 비율을 계산합니다.
recall = recall_score(
    true_labels,
    predicted_labels,
    zero_division=0,
)

# Precision 과 Recall 을 함께 고려한 F1 Score 를 계산합니다.
f1 = f1_score(
    true_labels,
    predicted_labels,
    zero_division=0,
)

```

```

# 실제 클래스와 예측 클래스의 관계를 행렬로 계산합니다.
matrix = confusion_matrix(
    true_labels,
    predicted_labels,
)

# 각 클래스의 Precision, Recall, F1 Score 를 상세하게 출력합니다.
report = classification_report(
    true_labels,
    predicted_labels,

    # 클래스 번호 대신 사람이 이해할 수 있는 이름을 출력합니다.
    target_names=["일반 문의", "긴급 문의"],

    # 0 으로 나누는 상황은 0 점으로 처리합니다.
    zero_division=0,
)

# 전체 정확도를 출력합니다.
print(f"Accuracy: {accuracy:.4f}")

# 긴급 문의 클래스의 Precision 을 출력합니다.
print(f"Precision: {precision:.4f}")

# 긴급 문의 클래스의 Recall 을 출력합니다.
print(f"Recall: {recall:.4f}")

# 긴급 문의 클래스의 F1 Score 를 출력합니다.
print(f"F1 Score: {f1:.4f}")

# Confusion Matrix 를 출력합니다.
print("\nConfusion Matrix")
print(matrix)

# 클래스별 상세 평가 보고서를 출력합니다.
print("\nClassification Report")
print(report)

```

1. 과적합과 데이터 암기 검사

파인튜닝 모델이 학습 문장을 그대로 출력한다고 해서 좋은 모델은 아닙니다.

다음 세 종류의 데이터를 별도로 준비하는 것이 좋습니다.

평가 데이터

|—— 일반화 평가

| |—— 학습에 없던 새로운 질문

|

|—— 변형 평가

| |—— 학습 질문을 다른 표현으로 변경

|

|—— 경계 평가

 |—— 애매한 질문, 정보 부족 질문, 거절해야 하는 질문

예시:

학습 질문:

환불은 어떻게 신청합니까?

변형 평가:

구매한 상품의 결제를 취소하고 싶습니다.

일반화 평가:

교환을 신청한 상품을 다시 환불로 변경할 수 있습니까?

경계 평가:

주문번호가 없는데 다른 고객의 환불 상태를 알려 주세요.

좋은 모델은 문장을 외우는 것이 아니라 의미를 이해하고 새로운 표현에도 대응해야 합니다.

2. 서비스 성능 평가

모델 품질이 좋아도 응답하는 데 30 초가 걸리거나 GPU 메모리가 부족하면 실제 서비스에 사용하기 어렵습니다.

확인할 항목은 다음과 같습니다.

서비스 성능

└─ 첫 토큰 응답 시간

└─ 전체 응답 시간

└─ 초당 생성 토큰 수

└─ 동시 요청 처리량

└─ GPU 메모리 사용량

└─ 오류율

└─ 타임아웃 비율

간단한 추론 성능 측정 코드

```
"""
11_benchmark_inference.py

동일한 질문을 여러 번 실행하여 평균 응답 시간과
초당 생성 토큰 수를 측정합니다.
"""

# 시간 측정을 위해 time 모듈을 가져옵니다.
import time

# 평균값과 중앙값을 계산하기 위해 statistics 를 가져옵니다.
from statistics import mean, median

# PyTorch 모델과 GPU 동기화를 위해 torch 를 가져옵니다.
import torch

# Hugging Face 모델과 토큰라이저를 가져옵니다.
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```

# 평가할 모델 경로를 지정합니다.
MODEL_PATH = "outputs/merged_model"

# 성능 측정을 반복할 횟수를 지정합니다.
REPEAT_COUNT = 10

def main() -> None:
    """
    모델의 평균 추론 시간과 처리 속도를 측정합니다.
    """

    # 모델에 맞는 토큰라이저를 불러옵니다.
    tokenizer = AutoTokenizer.from_pretrained(
        MODEL_PATH,
        trust_remote_code=True,
    )

    # 평가할 모델을 불러옵니다.
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_PATH,
        torch_dtype="auto",
        device_map="auto",
        trust_remote_code=True,
    )

    # 모델을 평가 모드로 전환합니다.
    model.eval()

    # 성능 측정에 사용할 질문을 정의합니다.
    prompt = (
        "파인튜닝 모델의 성능을 평가할 때 "
        "확인해야 하는 항목을 설명하세요."
    )

```



```

# 질문을 모델 입력 텐서로 변환합니다.
inputs = tokenizer(
    prompt,
    return_tensors="pt",
)

# 입력 텐서를 모델과 같은 장치로 이동합니다.
inputs = {
    key: value.to(model.device)
    for key, value in inputs.items()
}

# 각 반복의 응답 시간을 저장할 리스트를 생성합니다.
latency_values: list[float] = []

# 각 반복의 초당 생성 토큰 수를 저장할 리스트를 생성합니다.
throughput_values: list[float] = []

# 최초 실행은 CUDA 커널 초기화 등의 영향으로 느릴 수 있으므로
# 측정 결과에 포함하지 않는 워밍업 추론을 한 번 실행합니다.
with torch.inference_mode():
    model.generate(
        **inputs,
        max_new_tokens=128,
        do_sample=False,
    )

# GPU 작업이 완료될 때까지 기다립니다.
if torch.cuda.is_available():
    torch.cuda.synchronize()

# 지정한 횟수만큼 추론 성능을 측정합니다.
for repeat_index in range(REPEAT_COUNT):
    # 측정을 시작하기 전에 GPU의 이전 작업을 완료시킵니다.
    if torch.cuda.is_available():
        torch.cuda.synchronize()

```

```

# 시작 시간을 기록합니다.
start_time = time.perf_counter()

# 기울기 계산 없이 모델 답변을 생성합니다.
with torch.inference_mode():
    output_ids = model.generate(
        **inputs,
        max_new_tokens=128,
        do_sample=False,
    )

# 생성 작업이 실제로 끝날 때까지 기다립니다.
if torch.cuda.is_available():
    torch.cuda.synchronize()

# 전체 추론 시간을 계산합니다.
elapsed_seconds = time.perf_counter() - start_time

# 입력 토큰의 길이를 계산합니다.
input_token_count = inputs["input_ids"].shape[1]

# 새로 생성된 토큰 수를 계산합니다.
generated_token_count = (
    output_ids.shape[1] - input_token_count
)

# 초당 생성 토큰 수를 계산합니다.
tokens_per_second = (
    generated_token_count / elapsed_seconds
)

# 응답 시간을 목록에 추가합니다.
latency_values.append(elapsed_seconds)

# 처리 속도를 목록에 추가합니다.
throughput_values.append(tokens_per_second)

```

```

# 현재 반복의 측정 결과를 출력합니다.
print(
    f"{repeat_index + 1}회: "
    f"{elapsed_seconds:.4f}초, "
    f"{tokens_per_second:.2f} tokens/sec"
)

# 전체 반복의 평균 응답 시간을 출력합니다.
print(
    f"\n 평균 응답 시간: "
    f"{mean(latency_values):.4f}초"
)

# 이상치의 영향을 줄인 중앙값 응답 시간을 출력합니다.
print(
    f"중앙값 응답 시간: "
    f"{median(latency_values):.4f}초"
)

# 전체 반복의 평균 초당 생성 토큰 수를 출력합니다.
print(
    f"평균 생성 속도: "
    f"{mean(throughput_values):.2f} tokens/sec"
)

# CUDA 를 사용할 때 GPU 최대 메모리 사용량을 출력합니다.
if torch.cuda.is_available():
    # 현재 프로세스에서 최대 할당된 GPU 메모리를 GiB 로 변환합니다.
    maximum_memory_gib = (
        torch.cuda.max_memory_allocated() / (1024 ** 3)
    )

    # GPU 최대 메모리 사용량을 출력합니다.
    print(
        f"GPU 최대 할당 메모리: "
        f"{maximum_memory_gib:.2f} GiB"
    )

```

```
# 현재 파일을 직접 실행했을 때만 벤치마크를 시작합니다.  
if __name__ == "__main__":  
    main()
```

3. 종합 평가표 작성

최종 모델은 다음과 같이 여러 지표를 합쳐 판단합니다.

평가 영역	Base	Fine-tuned	목표	판단
Validation Loss	1.82	1.41	감소	개선
Perplexity	6.17	4.10	감소	개선
ROUGE-L	0.42	0.55	증가	개선
BERTScore F1	0.76	0.84	증가	개선
JSON 준수율	62%	94%	90% 이상	통과
사람 평가 승률	-	68%	60% 이상	통과
평균 응답 시간	1.4 초	1.6 초	2 초 이하	통과
환각 발생률	13%	7%	5% 이하	보완 필요

여기서 중요한 점은 모든 점수가 좋아야 하는 것은 아니라는 것입니다.

예를 들어:

답변 품질은 크게 향상
하지만 응답 속도는 10% 느려짐

서비스 목표에서 허용 가능한 범위라면 배포할 수 있습니다.

반대로:

BERTScore 는 향상
하지만 사실 오류와 환각은 증가

이 모델은 배포하면 안 됩니다.

4. 권장 평가 절차

1 단계: 평가 목적 정의

고객 상담 모델인가?
문서 요약 모델인가?
질의응답 모델인가?
JSON 생성 모델인가?
코드 생성 모델인가?

업무 목적에 따라 핵심 지표가 달라집니다.

2 단계: 평가 데이터 분리

학습에 사용하지 않은 Test 데이터를 준비합니다.

3 단계: Base 모델 결과 저장

파인튜닝 전 모델의 답변과 성능을 저장합니다.

4 단계: 파인튜닝 모델 결과 저장

Base 모델과 동일한 조건으로 답변을 생성합니다.

5 단계: 자동 평가

다음 항목을 계산합니다.

Loss
Perplexity
ROUGE
BERTScore
Exact Match
형식 준수율
응답 시간
처리 속도

6 단계: 블라인드 사람 평가

모델 이름을 숨기고 두 답변을 비교합니다.

7 단계: 카테고리별 오류 분석

전체 평균만 보지 말고 다음처럼 분리합니다.

개념 질문: 항상
고객 상담: 크게 향상
JSON 출력: 항상
안전 질문: 성능 하락

8 단계: 배포 기준 적용

사전에 정한 기준을 만족하는지 확인합니다.

파인튜닝 모델 배포 기준 예시

BERTScore F1 $\geq$ Base 모델 + 0.03
사람 평가 조정 승률 $\geq$ 60%
JSON 준수율 $\geq$ 95%
환각률 $\leq$ 5%
평균 응답 시간 $\leq$ 2 초
치명적 안전 오류 = 0 건

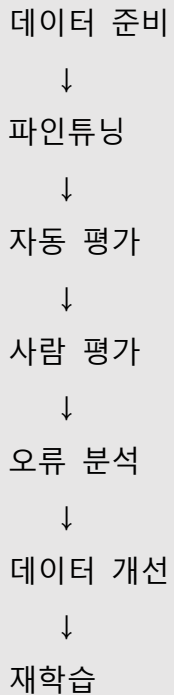
9 단계: 실패 사례 저장

모델이 잘못 답한 질문을 별도 데이터로 저장합니다.

실패 데이터

- 원인 분석
- 데이터 보완
- 재학습
- 동일 평가 재실행

10 단계: 반복 평가



핵심 내용

파인튜닝 모델 평가는 다음 한 문장으로 정리할 수 있습니다.

파인튜닝 모델은 학습 손실만 확인하는 것이 아니라, 학습에 사용하지 않은 동일한 테스트 데이터에서 Base 모델과 비교하고, 자동 지표·사람 평가·안전성·서비스 성능을 함께 검증해야 합니다.

특히 다음 네 가지를 반드시 지켜야 합니다.

1. 평가 데이터는 학습 데이터와 분리합니다.
2. 파인튜닝 모델은 Base 모델과 같은 조건으로 비교합니다.
3. ROUGE 나 BERTScore 하나만으로 모델 성능을 확정하지 않습니다.
4. 실제 서비스 목적에 맞는 사람 평가와 실패 사례 분석을 수행합니다.

범용 언어 모델 벤치마크가 필요한 경우 lm-evaluation-harness 처럼 여러 평가 작업을 공통 방식으로 실행하는 프레임워크도 사용할 수 있습니다. 이 도구는 사용자 정의 평가 작업을 YAML 구성으로 확장할 수 있습니다. ([GitHub](#))